disparate systems with different communication patterns.

Use of OpenFeign

In this project, OpenFeign has been integrated as a declarative REST client to simplify inter-service communication between the microservices within the Micro Job App architecture. Instead of manually using RestTemplate or WebClient, Feign clients have been defined via interfaces annotated with @FeignClient. This allows different microservices (such as the Job Service, User Service, or Notification Service) to easily call each other over HTTP without writing boilerplate code for request handling.

For example, when the Job Service needs to fetch user profile data from the User Service, it simply calls a method on the corresponding Feign interface, and OpenFeign automatically handles the request, response mapping, and error handling under the hood. This greatly enhances readability, maintainability, and modularity of the code.

Listing 2: Feign client example

```
@FeignClient(name = "user-service")
public interface UserClient { @
GetMapping("/users/{id}")
UserDTO getUserById(@PathVariable("id")
Long id);
}
```

Use of Zipkin

Zipkin has been incorporated into this project for distributed tracing across the microservices, helping developers track and visualise the flow of requests through the system. Each microservice is configured to send tracing data to the Zipkin server, which collects and aggregates these traces. When a client initiates a request (for example, posting a new job), the system records detailed timing information as the request travels through multiple services (such as Job Service → Notification Service → Email Service). This helps in identifying performance bottlenecks, understanding request latency, and diagnosing failures across service boundaries. The integration with Spring Cloud Sleuth further enriches the tracing data by automatically tagging logs with trace IDs and span IDs, making it easier to correlate logs and traces.

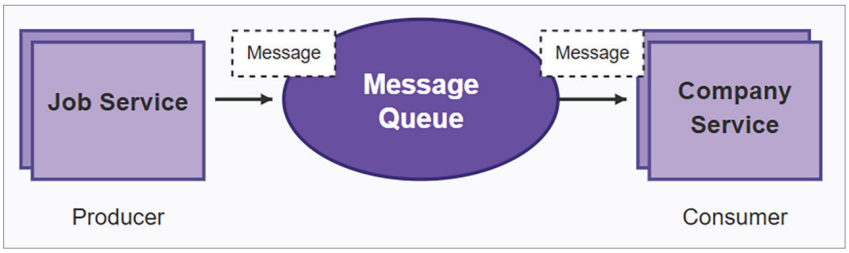

Figure 9: Flow of messages

Listing 3: Zipkin configuration

```
spring.zipkin.base-url=http://
localhost:9411/ spring.sleuth.sampler.
probability=1.0
```

The results

The transition from a monolithic architecture to a microservices-based architecture for the job portal resulted in significant improvements across multiple dimensions of system performance, development agility, and operational efficiency. Here are the key benefits we observed.

Before migration, the monolithic application could reliably handle up to 500 concurrent users before response times degraded significantly (average latency increased to over 1200ms). After migration, the microservices-based system handled 2000 concurrent users with acceptable performance (average latency stayed below 400ms). This represents a 4X increase in user capacity with a 66% reduction in average response time. The experimental setup and results are demonstrated in Figures 10 and 11.

Deployment time: The full deployment of the monolithic application took approximately 30 minutes with downtime. In the microservices setup, deployment of individual services (Job, Company, and Review) now takes about 5 minutes per service with zero downtime using rolling updates. This resulted in an 83% reduction in deployment time.

Fault isolation and recovery: Previously, a failure in one module could cause partial or total system downtime. With microservices, failures are isolated; for instance, issues in the Review service do not affect Job or Company services. Average recovery time was reduced from around 20 minutes to about 3 minutes due to independent service restarts.

Resource utilisation: Monolithic deployment required 4 vCPUs and 8GB RAM constantly, irrespective of module loads. In contrast, microservices scaled independently:

- **Job service:** 2 vCPUs, 4GB RAM
- **Company service:** 1 vCPU, 2GB RAM
- **Review service:** 1 vCPU, 2GB RAM

This led to a 30–40% reduction in total resource consumption under normal load conditions.

Improvements and challenges

The migration to a microservices architecture significantly enhanced the flexibility, performance, and reliability of the job portal system. By decoupling services, implementing asynchronous communication (RabbitMQ), enabling service discovery (Eureka), utilising declarative REST clients (OpenFeign), and adopting distributed tracing (Zipkin), the system now supports dynamic scaling, fault isolation, and faster feature rollouts.